Assignment 3

React — Hooks, Components & Routing: Questions & Answers

Question 1

Write a functional React component `Counter` using `useState` that increments and decrements a number on button clicks. In 1 line, explain why a plain JavaScript variable (instead of useState) would not update the UI.

Answer:

```
import React, { useState } from 'react';

function Counter() {
  const [count, setCount] = useState(0);

  return (
    <div>
      <h3>Count: {count}</h3>
      <button onClick={() => setCount(count + 1)}>Increment</button>
      <button onClick={() => setCount(count - 1)}>Decrement</button>
    </div>
  );
}

export default Counter;
```

Why a plain JavaScript variable fails:

Modifying a plain JavaScript variable does not trigger React's re-render mechanism to update the DOM UI.

Question 2

This component calls `setCount` directly inside the render body with no condition, causing an infinite re-render loop. Fix the code.

```
function Counter() {
  const [count, setCount] = useState(0);
  setCount(count + 1);
  return <p>{count}</p>;
}
```

Fixed Code:

```
import React, { useState } from 'react';

function Counter() {
  const [count, setCount] = useState(0);

  return (
    <div>
      <p>{count}</p>
      <button onClick={() => setCount(count + 1)}>Increment</button>
    </div>
  );
}
```

Question 3

Write a `useEffect` hook that fetches data from `/api/posts` when the component first mounts. In 1 line, explain why the empty dependency array `[]` is necessary here.

Answer:

```
import React, { useState, useEffect } from 'react';

function Posts() {
  const [posts, setPosts] = useState([]);

  useEffect(() => {
    fetch('/api/posts')
      .then(res => res.json())
      .then(data => setPosts(data))
      .catch(err => console.error(err));
  }, []); // Empty dependency array

  return (
    <ul>
      {posts.map(post => <li key={post.id}>{post.title}</li>)}
    </ul>
  );
}
```

Why empty [] is necessary:

The empty array ensures the effect runs only once when the component mounts, preventing infinite fetch loops on re-renders.

Question 4

In this code, two sibling components each keep their own local copy of the same value, so they fall out of sync. Refactor the code to lift the shared state up to the parent component.

```
function PriceInput() {
  const [price, setPrice] = useState(0);
  return <input value={price} onChange={e => setPrice(e.target.value)} />;
}
function PriceDisplay() {
  const [price, setPrice] = useState(0);
  return <p>Price: {price}</p>;
}
```

Answer:

```
import React, { useState } from 'react';

// Parent Component holding shared state
function ParentComponent() {
  const [price, setPrice] = useState(0);

  return (
    <div>
      <PriceInput price={price} setPrice={setPrice} />
      <PriceDisplay price={price} />
    </div>
  );
}

function PriceInput({ price, setPrice }) {
  return <input value={price} onChange={e => setPrice(e.target.value)} />;
}

function PriceDisplay({ price }) {
  return <p>Price: {price}</p>;
}
```

Question 5

Write a custom React hook `useLocalStorage(key, initialValue)` that behaves like useState but also saves and reads the value from localStorage.

Answer:

```
import { useState, useEffect } from 'react';

function useLocalStorage(key, initialValue) {
  // Get stored value or fallback to initialValue
  const [storedValue, setStoredValue] = useState(() => {
    try {
      const item = window.localStorage.getItem(key);
      return item ? JSON.parse(item) : initialValue;
    } catch (error) {
      console.error(error);
      return initialValue;
    }
  });

  // Sync state changes with localStorage
  useEffect(() => {
    try {
      window.localStorage.setItem(key, JSON.stringify(storedValue));
    } catch (error) {
      console.error(error);
    }
  }, [key, storedValue]);

  return [storedValue, setStoredValue];
}

export default useLocalStorage;
```

Question 6

This list renders correctly, but React logs a "key" warning and re-renders inefficiently. Fix the code.

```
{items.map(item => (
  <li>{item.name}</li>
))}
```

Fixed Code:

```
{items.map(item => (
  <li key={item.id}>{item.name}</li>
))}
```

(Note: If `item.id` doesn't exist, use another unique identifier, or fall back to the index as a last resort.)

Question 7

Write a reusable React <Button> component that accepts a `variant` prop ("primary" or "danger") and applies a different CSS class depending on its value.

Answer:

```
import React from 'react';

function Button({ variant = 'primary', children, onClick }) {
  const className = variant === 'danger' ? 'btn-danger' : 'btn-primary';

  return (
    <button className={`btn ${className}`} onClick={onClick}>
      {children}
    </button>
  );
}

export default Button;
```

Question 8

In this app, `ChildCard` re-renders every time `ParentDashboard` re-renders, even when its own props haven't changed. Explain briefly why this happens, then fix it using `React.memo`.

```
function ParentDashboard() {
  const [tick, setTick] = useState(0);
  return (
    <div>
      <button onClick={() => setTick(tick + 1)}>Refresh</button>
      <ChildCard title="Static Title" />
    </div>
  );
}
function ChildCard({ title }) {
  console.log('ChildCard rendered');
  return <h3>{title}</h3>;
}
```

Why it happens:

By default, when a parent component re-renders in React, all of its child components re-render automatically regardless of whether their props changed.

Fixed Code:

```
import React, { useState, memo } from 'react';

// Wrap ChildCard with React.memo to skip re-renders if props don't change
const ChildCard = memo(function ChildCard({ title }) {
  console.log('ChildCard rendered');
  return <h3>{title}</h3>;
});

function ParentDashboard() {
  const [tick, setTick] = useState(0);
  return (
    <div>
      <button onClick={() => setTick(tick + 1)}>Refresh</button>
      <ChildCard title="Static Title" />
    </div>
  );
}

export default ParentDashboard;
```

Question 9

Write React Router (v6) code to set up 3 routes — "/", "/products", and "/products/:id" — including a working <Link> for each in the navbar.

Answer:

```
import React from 'react';
import { BrowserRouter, Routes, Route, Link, useParams } from 'react-router-dom';

function Home() { return <h2>Home Page</h2>; }
function Products() { return <h2>Products List</h2>; }
function ProductDetails() {
  const { id } = useParams();
  return <h2>Product ID: {id}</h2>;
}

function App() {
  return (
    <BrowserRouter>
      <nav>
        <Link to="/">Home</Link> |
        <Link to="/products">Products</Link> |
        <Link to="/products/1">Product 1</Link>
```

```
      </nav>

      <Routes>
        <Route path="/" element={<Home />} />
        <Route path="/products" element={<Products />} />
        <Route path="/products/:id" element={<ProductDetails />} />
      </Routes>
    </BrowserRouter>
  );
}

export default App;
```

Question 10

This form has a separate `useState` for every field, making the code repetitive. Refactor it into a single state object with one shared `onChange` handler.

```
const [name, setName] = useState('');
const [email, setEmail] = useState('');
const [phone, setPhone] = useState('');
```

Answer:

```
import React, { useState } from 'react';

function UserForm() {
  const [formData, setFormData] = useState({
    name: '',
    email: '',
    phone: ''
  });

  // Shared onChange handler
  const handleChange = (e) => {
    const { name, value } = e.target;
    setFormData(prev => ({
      ...prev,
      [name]: value
    }));
  };

  return (
    <form>
      <input name="name" value={formData.name}
             onChange={handleChange} placeholder="Name" />
      <input name="email" value={formData.email}
             onChange={handleChange} placeholder="Email" />
      <input name="phone" value={formData.phone}
             onChange={handleChange} placeholder="Phone" />
    </form>
  );
}

export default UserForm;
```